

Procedural programming

Functional programming

Object Oriented Programming

- ✔

Video: Introduction to Object Oriented Programming

5 min
- ✔

Reading: OOP Principles

20 min
- ✔

Video: Python classes and instances

4 min
- ✔

Reading: Exercise: Define a Class

30 min
- ✔

Reading: Define a Class - solution

10 min
- 📋

Practice Quiz: Self-review: Define a Class

4 questions
- ✔

Video: Instantiate a custom Object

4 min
- ✔

Reading: Exercise: Instantiate a custom Object

30 min
- ✔

Reading: Instantiate a custom Object - solution

10 min
- 📋

Practice Quiz: Self-review: Instantiate a custom Object

4 questions
- ✔

Video: Instance methods

4 min
- ✔

Video: Parent classes vs. child classes

6 min
- ✔

Reading: Inheritance and Multiple Inheritance

30 min
- ✔

Reading: Exercise: Classes and object exploration

30 min
- 📄

Video: Abstract classes and methods

4 min
- 📋

Practice Quiz: Abstract classes and methods

5 questions
- 📋

Programming Assignment: Abstract Classes and Methods

3h
- 📄

Video: Method Resolution Order

5 min
- 📋

Reading: Working with Methods: Examples

20 min
- 📋

Reading: Exercise: Working with Methods

30 min
- 📋

Reading: Working with Methods - solution

10 min
- 📋

Practice Quiz: Self-review: Working with Methods

6 questions
- 📄

Video: Module summary: Programming paradigms

2 min
- 📋

Quiz: Module quiz: Programming Paradigms

8 questions
- 📋

Reading: Additional resources

5 min

# Exercise: Classes and object exploration

In this reading, you will explore the behaviour of functions, objects and classes in Python and how the flow of execution of different program statements works to enable better understanding.

You will perform minor modifications in the given code to observe how it changes the output.

First, set up a file called `class_explore.py` that contains the following piece of code. Alternatively, you can use the interactive environment here.

```
1 class A:
2     def __init__(self, c):
3         print("-----Inside class A-----")
4         self.c = c
5         print("Print inside A.")
6
7     def alpha(self):
8         c = self.c + 1
9         return c
10
11 print(dir(A))
12 print("Instantiating A..")
13 a = A(1)
14 print(a.alpha())
15
16 class B:
17     def __init__(self, a):
18         print("-----Inside class B-----")
19         self.a = a
20
21     print(a.alpha())
22     d = 5
23     print(d)
24     print(a)
25
26 print("Instantiating B..")
27 b = B(a)
28 print(a)
```

Run

Reset

```
Hello World!
Print inside A.
['_class_', '_delattr_', '_dict_', '_dir_', '_doc_', '_eq_', '_format_', '_ge_', '_getattr_', '_gt_', '_hash_', '_init_', '_init_subclass_', '_le_', '_lt_', '_module_', '_ne_', '_new_', '_reduce_', '_repr_', '_setattr_', '_sizeof_', '_str_', '_subclass_', '_weakref_', '_xattrs_']
Instantiating A.
-----Inside class A-----
2
2
5
<submission.A object at 0x7f95fe890be0>
Instantiating B..
-----Inside class B-----
<submission.A object at 0x7f95fe890be0>
```

Now, modify the code as per the instructions below and observe the changes.

**Step 1: Run the code and observe its output. Take note of every line in the output and how it is different from the output you expected.**

Algorithmically we can view the program consisting of the following:

- Class definition of A
  - 1.1 Constructor for A
  - 1.2 Definition of local method alpha()
- Instantiating object a over class A
- Calling method alpha() over object of class A
- Class definition of B
- Constructor for B
  - 5.1 Calling method alpha() over object of class A
  - 5.2 Instantiating object a over class B \*.

Additional print statements distributed through the code.

**Step 2: Comment out lines #13, 14, 21, 24, 27 and 28. Run the code again.**

The output is:

```
1 Hello World!
2 Print inside A.
3 ['_class_', '_delattr_', '_dict_', '_dir_', '_doc_', '_eq_', '_format_', '_ge_', '_getattr_', '_gt_', '_hash_', '_init_', '_init_subclass_', '_le_', '_lt_', '_module_', '_ne_', '_new_', '_reduce_', '_repr_', '_setattr_', '_sizeof_', '_str_', '_subclass_', '_weakref_', '_xattrs_']
4 Instantiating A..
5 5
6 Instantiating B..
```

Even if you have commented out the creation of instances for both classes A and B, the output still shows "Print inside A" and "Print inside B" and also the value of variable d, which is 5. How is that possible?

It's because statements inside a class body get executed irrespective of the instance creation. You will also see how the print statement "Inside class A", which is inside the constructor, is not executed because it's inside a function.

The value of d=5 being printed demonstrates that the namespace and scope of the variable is determined by the interpreter before you create any instance of the class or call any function inside it. If you observe the list you get by calling the dir() function, you'll note that the last entry is the alpha() function added to the namespace of A.

**Step 3: Now remove the comment for lines 21 and 24.**

If you run the code at this point, it will throw an error, "NameError: name 'a' is not defined". Take note of how you have passed the object a to the constructor of class B and the code still worked fine earlier. Only when you tried to 'use' object a, did you get an error because it has not been instantiated. In other words, Python still does not know what 'a' means. The same will happen if you remove commenting next to line 28.

**To make the code work, now remove the # in front of line 14 and run it again.**

The output is:

```
1 Hello World!
2 Print inside A.
3 ['_class_', '_delattr_', '_dict_', '_dir_', '_doc_', '_eq_', '_format_', '_ge_', '_getattr_', '_gt_', '_hash_', '_init_', '_init_subclass_', '_le_', '_lt_', '_module_', '_ne_', '_new_', '_reduce_', '_repr_', '_setattr_', '_sizeof_', '_str_', '_subclass_', '_weakref_', '_xattrs_']
4 Instantiating A..
5 -----Inside class A-----
6 2
7 5
8 <submission.A object at 0x7fcab3ef6940>
9 Instantiating B..
```

**Step 4: Remove the commenting for line 27 and 28.**

The variable c of class A is modified over object a inside class B. Even though the instance of class B is still not created, the value of a.c is still getting updated, even outside the class, as evidenced by the final line in the output which shows the output is 2.

**Step 5: Finally remove all the remaining comments and run the code one more time.**

Here are a few observations.

- When you try and print the 'object' of class A as in lines 21 and 28, you get the address of the object instead of the contents.
- Note how the address of object a is the same both inside class B and in the global scope of the program. It remains the same irrespective of from where it is called.
- The alpha() function is called twice in the program, but you still get the output as 2 every time and not 3. That's because the value is updated only temporarily and not assigned to anything.

Revise items about classes, function calls and scope in case of confusion.

✔ Completed Go to next item

👍 Like 🗨️ Dislike 📄 Report an issue

